

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
```

```
data= pd.read_csv('dataset.csv')

df=pd.DataFrame(data)

df.head(10)
```

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_
0	30669	Male	3.0	0	0	No	children	Rural	95.12	18.0	
1	30468	Male	58.0	1	0	Yes	Private	Urban	87.96	39.2	never s
2	16523	Female	8.0	0	0	No	Private	Urban	110.89	17.6	
3	56543	Female	70.0	0	0	Yes	Private	Rural	69.04	35.9	formerly s
4	46136	Male	14.0	0	0	No	Never_worked	Rural	161.28	19.1	
5	32257	Female	47.0	0	0	Yes	Private	Urban	210.95	50.1	
6	52800	Female	52.0	0	0	Yes	Private	Urban	77.59	17.7	formerly s
7	41413	Female	75.0	0	1	Yes	Self-employed	Rural	243.53	27.0	never s
8	15266	Female	32.0	0	0	Yes	Private	Rural	77.67	32.3	s
9	28674	Female	74.0	1	0	Yes	Self-employed	Urban	205.84	54.6	never s

Next steps:

Generate code with df

New interactive sheet

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 43400 entries, 0 to 43399
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   id                    43400 non-null int64  
1   gender               43400 non-null object 
2   age                  43400 non-null float64
3   hypertension         43400 non-null int64  
4   heart_disease        43400 non-null int64  
5   ever_married         43400 non-null object 
6   work_type            43400 non-null object 
7   Residence_type       43400 non-null object 
8   avg_glucose_level    43400 non-null float64
9   bmi                  41938 non-null float64
10  smoking_status       30108 non-null object 
11  stroke               43400 non-null int64  
dtypes: float64(3), int64(4), object(5)
memory usage: 4.0+ MB
```

df.duplicated().sum()

np.int64(0)

df.shape #عدد الصفوف والاعمدة

(43400, 12)

df.isna().sum() #فحص القيم المفقودة

	0
id	0
gender	0
age	0
hypertension	0
heart_disease	0
ever_married	0
work_type	0
Residence_type	0
avg_glucose_level	0
bmi	1462
smoking_status	13292
stroke	0

dtype: int64

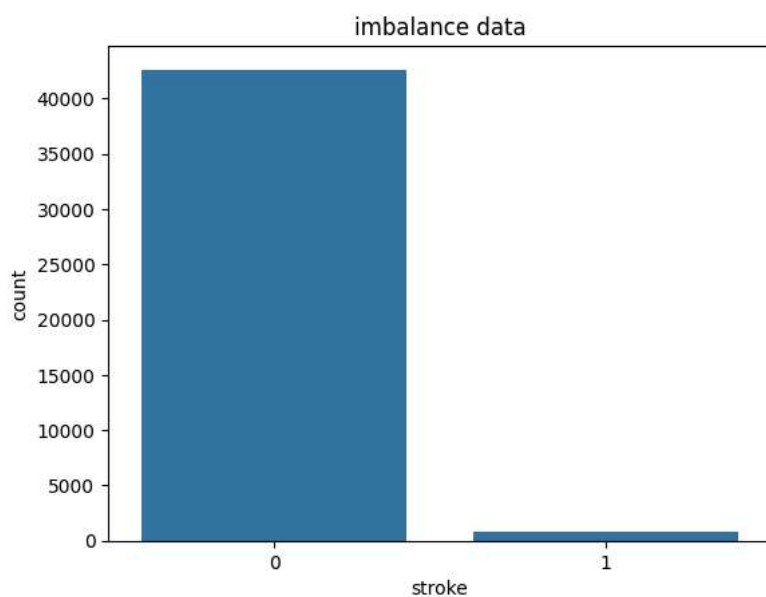
df['stroke'].value_counts()

	count
stroke	
0	42617
1	783

dtype: int64

تصوير توزيع البيانات

```
sns.countplot(x='stroke',data=df)
plt.title("imbalance data")
plt.show()
```



```
df = pd.DataFrame(data)
df = df.drop(columns=['id'])
# 1. تنظيف القيم المفقودة
df['bmi'] = df['bmi'].fillna(df['bmi'].mean())
df['smoking_status'] = df['smoking_status'].fillna('Unknown')

# One-Hot Encoding
```

```

categorical_columns = ['gender', 'ever_married', 'work_type', 'Residence_type', 'smoking_status']
df = pd.get_dummies(df, columns=categorical_columns, drop_first=True, dtype=int)

# العمر
df['is_senior'] = (df['age'] > 60).astype(int)

df['smokes_and_has_hypertension'] = ((df['smoking_status_smokes'] == 1) & (df['hypertension'] == 1)).astype(int)
df['is_obese'] = (df['bmi'] > 30).astype(int)
df['is_diabetic'] = (df['avg_glucose_level'] >= 200).astype(int)
df['senior_hypertension'] = ((df['age'] > 60) & (df['hypertension'] == 1)).astype(int)
# التي نريد التنبؤ بها stroke نحذف
X = df.drop('stroke', axis=1)
y = df['stroke']

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# معالجة عدم التوازن
from imblearn.combine import SMOTETomek

smt = SMOTETomek(random_state=42)

X_train, y_train = smt.fit_resample(X_train, y_train)

print(y_train.value_counts())

```

```

stroke
0    34056
1    34056
Name: count, dtype: int64

```

```

from sklearn.preprocessing import StandardScaler
Scaler=StandardScaler()
X_train=Scaler.fit_transform(X_train)
X_test=Scaler.transform(X_test)

```

logistic regression model

```

from sklearn.linear_model import LogisticRegression

logistic = LogisticRegression(max_iter=1000, class_weight='balanced')

```

```
logistic.fit(X_train,y_train)
```

▼ **LogisticRegression** ⓘ ?

```
LogisticRegression(class_weight='balanced', max_iter=1000)
```

```

# توقع نتائج بيانات الاختبار
logistic.fit(X_train, y_train)

y_probs = logistic.predict_proba(X_test)[: , 1]

# تغيير العتبة
y_pred = (y_probs >= 0.15).astype(int)

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# حساب المقاييس
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
class_rep = classification_report(y_test, y_pred)

# طباعة النتائج
print(f"Accuracy: {accuracy}")
print("\nConfusion Matrix:")
print(conf_matrix)
print("\nClassification Report:")
print(class_rep)

```

```
Accuracy: 0.6618663594470046
```

```

Confusion Matrix:
[[5631 2888]

```

```
[ 47 114]]

Classification Report:
      precision    recall  f1-score   support

     0       0.99      0.66      0.79      8519
     1       0.04      0.71      0.07       161

 accuracy      0.66      8680
 macro avg     0.51      0.68      0.43      8680
 weighted avg   0.97      0.66      0.78      8680
```

```
from sklearn.tree import DecisionTreeClassifier
decision_tree = DecisionTreeClassifier()
```

```
decision_tree.fit(X_train, y_train)
```

```
▼ DecisionTreeClassifier ⓘ ?
DecisionTreeClassifier()
```

```
# توقع النتائج
y_pred_dt = decision_tree.predict(X_test)
```

```
# حساب المقاييس
accuracy_dt = accuracy_score(y_test, y_pred_dt)
conf_matrix_dt = confusion_matrix(y_test, y_pred_dt)
classification_rep_dt = classification_report(y_test, y_pred_dt)
# طباعة النتائج
print(f"Accuracy: {accuracy_dt}")
print("\nConfusion Matrix:")
print(conf_matrix_dt)
print("\nClassification Report:")
print(classification_rep_dt)
```

```
Accuracy: 0.9352534562211982
```

```
Confusion Matrix:
[[8101  418]
 [ 144   17]]
```

```
Classification Report:
      precision    recall  f1-score   support

     0       0.98      0.95      0.97      8519
     1       0.04      0.11      0.06       161

 accuracy      0.94      8680
 macro avg     0.51      0.53      0.51      8680
 weighted avg   0.97      0.94      0.95      8680
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score
```

```
rf_model = RandomForestClassifier(
    n_estimators=200,
    class_weight='balanced',
    max_depth=10,
    min_samples_leaf=2,
    random_state=42
)
```

```
rf_model.fit(X_train, y_train)
```

```
▼ RandomForestClassifier ⓘ ?
RandomForestClassifier(class_weight='balanced', max_depth=10,
                       min_samples_leaf=2, n_estimators=200, random_state=42)
```

```
# استخدام الموديل المدرب لتوقع النتائج من بيانات الاختبار
y_pred = rf_model.predict(X_test)
```

```

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

accuracy_rf = accuracy_score(y_test, y_pred)
conf_matrix_rf = confusion_matrix(y_test, y_pred)
classification_rep_rf = classification_report(y_test, y_pred)

print(f"Accuracy: {accuracy_rf}")
print("\nConfusion Matrix:")
print(conf_matrix_rf)
print("\nClassification Report:")
print(classification_rep_rf)

```

Accuracy: 0.8885944700460829

Confusion Matrix:

```
[[7672  847]
 [ 120   41]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.98	0.90	0.94	8519
1	0.05	0.25	0.08	161
accuracy			0.89	8680
macro avg	0.52	0.58	0.51	8680
weighted avg	0.97	0.89	0.92	8680

```

neg = (y_train == 0).sum()
pos = (y_train == 1).sum()
scale = neg / pos

xgb_model = XGBClassifier(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=5,
    scale_pos_weight=scale * 2,
    random_state=42
)

xgb_model.fit(X_train, y_train)

```

▼ XGBClassifier ⓘ ?

```

XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=None, device=None, early_stopping_rounds=None,
               enable_categorical=True, eval_metric=None, feature_types=None,
               feature_weights=None, gamma=None, grow_policy=None,
               importance_type=None, interaction_constraints=None,
               learning_rate=0.05, max_bin=None, max_cat_threshold=None,
               max_cat_to_onehot=None, max_delta_step=None, max_depth=5,
               max_leaves=None, min_child_weight=None, missing=nan,
               monotone_constraints=None, multi_strategy=None, n_estimators=200,
               n_jobs=None, num_parallel_tree=None, ...)

```

```

# توقع النتائج
y_pred_xgb = xgb_model.predict(X_test)

```

```

# حساب المقاييس
accuracy_xgb = accuracy_score(y_test, y_pred_xgb)
conf_matrix_xgb = confusion_matrix(y_test, y_pred_xgb)
classification_rep_xgb = classification_report(y_test, y_pred_xgb)

```

```

print(f"Accuracy: {accuracy_xgb}")
print("\nConfusion Matrix:")
print(conf_matrix_xgb)
print("\nClassification Report:")
print(classification_rep_xgb)

```

Accuracy: 0.8512672811059908

Confusion Matrix:

```
[[7338 1181]
 [ 110   51]]
```

```
Classification Report:
              precision    recall  f1-score   support

     0       0.99        0.86        0.92        8519
     1       0.04        0.32        0.07         161

 accuracy          0.85        8680
 macro avg         0.51        0.59        0.50        8680
 weighted avg      0.97        0.85        0.90        8680
```

```
from IPython.display import HTML
HTML('<div style="page-break-before: always;"></div>')
```

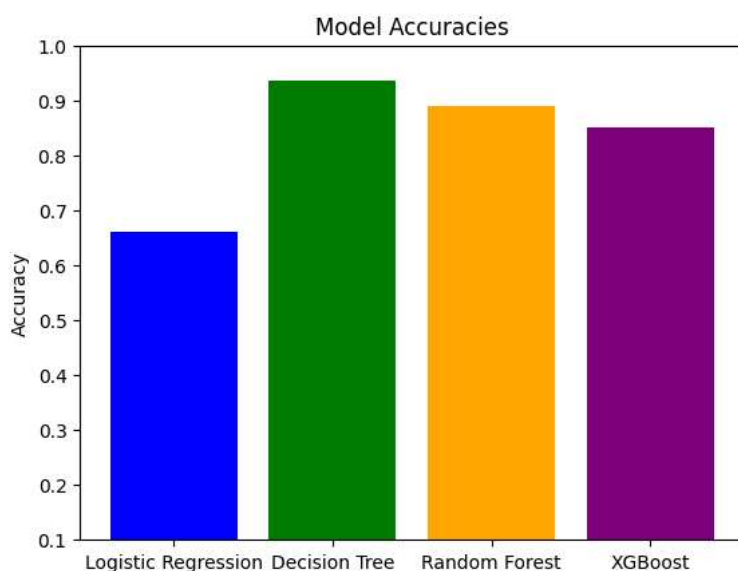
مقارنة دقة النماذج

```
import matplotlib.pyplot as plt

models = ['Logistic Regression', 'Decision Tree', 'Random Forest', 'XGBoost']

accuracies = [accuracy, accuracy_dt, accuracy_rf, accuracy_xgb]
plt.bar(models, accuracies, color=['blue', 'green', 'orange', 'purple'])

plt.ylabel('Accuracy')
plt.title('Model Accuracies')
plt.ylim(0.1, 1.0)
plt.show()
```



حفظ النموذج

```
xgb_model.fit(X_train, y_train)
xgb_model.save_model('stroke_model.json')

with open('scaler.sav', 'wb') as f:
    pickle.dump(Scaler, f)
```

